

CS 610 Semester 2024–2025-I: Assignment 1

Name: **Vinayak Devvrat**

Roll Number: **241110079**

August 24, 2024

Problem 1

Array Size = $2^{10} * 2^5 * 2^2 = 2^{17}$ B ; Cache Size = 2^{18} B

Since, array size < cache size, i.e., the cache is large enough to store the array, this would imply **conflict miss = capacity miss = 0**. The cold misses will occur only in the very first iteration of the loop.

The following table indicates the access pattern, total number of accesses and misses for the given stride value. Number of memory accesses is calculated as : $\lceil \frac{ArraySize}{Stride} \rceil$

S No.	Stride	Hit/Miss Pattern	Memory Ac-cesses	Remarks	Total Misses
1	1	A[0] (M), A[1] (H), A[2] (H) ... A[16] (M), A[17] (H), A[18] (H) ... A[32] (M), ...	2^{15}	Every 16th access will be a miss.	$\frac{2^{15}}{16} = 2^{11} = \mathbf{2048}$
2	4	A[0] (M), A[4] (H), A[8] (H) ... A[16] (M), A[20] (H), A[24] (H) ... A[32] (M), ...	2^{13}	Every 4th access will be a miss.	$\frac{2^{13}}{4} = 2^{11} = \mathbf{2048}$
3	16	A[0] (M), A[16] (M), A[32] (M), ...	2^{11}	Every access will be a miss.	$2^{11} = \mathbf{2048}$
4	64	A[0] (M), A[64] (M), A[128] (M), ...	2^9	Every access will be a miss.	$2^9 = \mathbf{512}$
5	2K	A[0] (M), A[2048] (M), A[4096] (M), ...	2^4	Every access will be a miss.	$2^4 = \mathbf{16}$
6	8K	A[0] (M), A[8192] (M), A[16384] (M), ...	2^2	Every access will be a miss.	$2^2 = \mathbf{4}$
7	16K	A[0] (M), A[16384] (M)	2^1	Every access will be a miss.	$2^1 = \mathbf{2}$

8	32K	A[0] (M)	1	Every access will be a miss.	1
---	-----	----------	---	------------------------------	---

Problem 2

Array Size = $1024 * 1024 * 1 = 2^{20}$ words; Cache Size = 2^{16} words.

Clearly, Cache Size < Array Size. Cache is not big enough to hold the entire array.

Block Size = BL = 8 words.

A. *kij* form FULLY ASSOCIATIVE CACHE

Table 1

	A	B	C
k	N	N	N
i	N	1	N
j	1	$\frac{N}{BL}$	$\frac{N}{BL}$
Total	N^2	$\frac{N^2}{BL}$	$\frac{N^3}{BL}$
	2^{20}	2^{17}	2^{27}

Considering the innermost loop **j**,

- **Array A:** As **j** is varied, the element accessed for A does not change. So, we have a factor of 1 here.
- **Array B:** It is a row major access so every BL access will be a miss. Hence, we get a factor of $\frac{N}{BL}$.
- **Array C:** Same as B, C is accessed in a row major order. Therefore we get a factor of $\frac{N}{BL}$.

For loop **i**,

- **Array A:** It is accessed in a column major order so every access will be a miss. Hence we get a factor of N.
- **Array B:** Accesses to B do not change as **i** is varied. Therefore, we get a factor of 1.
- **Array C:** It is also accessed in a column major order so every access will be a miss. Hence we get a factor of N.

For loop **k**, since the cache is not big enough to hold the array so as **k** is varied the values would need to be fetched again for all three arrays. Therefore, we have a factor of N for each case.

DIRECT MAPPED CACHE

Table 2

	A	B	C
k	N	N	N
i	N	N	N
j	1	$\frac{N}{BL}$	$\frac{N}{BL}$
Total	N^2	$\frac{N^3}{BL}$	$\frac{N^3}{BL}$
	2^{20}	2^{27}	2^{27}

Considering the innermost loop **j**, the factors for all arrays will remain the same as that for fully associative cache for the reasons stated previously.

For loop **i**,

- **Array A:** It is accessed in a column major order so every access will be a miss. Hence we get a factor of N.
- **Array B:** Accesses to B do not change as **i** is varied. But it is a direct mapped cache, so the same elements would have to be fetched again. Therefore, we get a factor of N.
- **Array C:** It is also accessed in a column major order so every access will be a miss. Hence we get a factor of N.

For loop **k**, the factors for all arrays will remain the same as that for fully associative cache for the reasons stated previously.

B. *jik* form

FULLY ASSOCIATIVE CACHE

Table 3

	A	B	C
j	N	$\frac{N}{BL}$	$\frac{N}{BL}$
i	N	1	N
k	$\frac{N}{BL}$	N	1
Total	$\frac{N^3}{BL}$	$\frac{N^2}{BL}$	$\frac{N^2}{BL}$
	2^{27}	2^{17}	2^{17}

Considering the innermost loop **k**,

- **Array A:** It is a row major access for A. So we get a factor of $\frac{N}{BL}$.

- **Array B:** B is accessed in a column major order. Hence, we get a factor of N.
- **Array C:** As **k** varies, accesses to C do not change. Hence, we get a factor of 1 here.

For loop **i**,

- **Array A:** It is accessed in a column major order so every access will be a miss. Hence we get a factor of N.
- **Array B:** Accesses to B do not change as **i** is varied. Therefore, we get a factor of 1.
- **Array C:** It is also accessed in a column major order so every access will be a miss. Hence we get a factor of N.

For loop **j**, since the cache is not big enough to hold the array so as **j** is varied the values would need to be fetched again for all three arrays. Therefore, we have a factor of N for each case.

DIRECT MAPPED CACHE

Table 4

	A	B	C
j	N	N	N
i	N	N	N
k	$\frac{N}{BL}$	N	1
Total	$\frac{N^3}{BL}$	N^3	N^2
	2^{27}	2^{30}	2^{20}

Considering the innermost loop **k**, the factors for all arrays will remain the same as that for fully associative cache for the reasons stated previously.

For loop **i**,

- **Array A:** It is accessed in a column major order so every access will be a miss. Hence we get a factor of N.
- **Array B:** Accesses to B do not change as **i** is varied. But it is a direct mapped cache, so the same elements would have to be fetched again. Therefore, we get a factor of N.
- **Array C:** It is also accessed in a column major order so every access will be a miss. Hence we get a factor of N.

For loop **j**, the factors for all arrays will remain the same as that for fully associative cache for the reasons stated previously.

Problem 3

Table 5

	A	X	y
k	N	N	1
j	N	$\frac{N}{BL}$	1
i	N	1	$\frac{N}{BL}$
Total	N^3	$\frac{N^2}{BL}$	$\frac{N}{BL}$
	2^{36}	2^{22}	2^{10}

Cache Size = 2^{24} B, Size of X = Size of A = $2^{12} * 2^{12} * 8 = 2^{27}$ B, Size of y = $2^{12} * 8 = 2^{15}$ B, $BL = \frac{32}{8} = 4$ words.

For innermost loop **i**,

- **Array A:** It is accessed in column major order. So, every access will be a miss. We get a factor of N.
- **Array X:** The access to X does not change as **i** varies. Hence we get a factor of 1.
- **Array y:** Since, Size of y < Cache Size, it will fit in the cache completely and we would only have cold misses associated with it. Hence we get a factor of $\frac{N}{BL}$.

For loop **j**,

- **Array A:** It is accessed in row major order but this is a direct mapped cache and it is not big enough so we would have to fetch elements again from memory. Hence, we get a factor of N.
- **Array X:** It is accessed in a row major order. So we get a factor of $\frac{N}{BL}$.
- **Array y:** It will have misses only during the first iteration of loop **i**. Hence we get a factor of 1.

For loop **k**,

- **Array A:** Since its size is more than that of the cache and the cache is direct mapped by the time a single **k** loop finishes, we would have to fetch again from memory. Hence we get a factor of N.
- **Array X:** It will be accessed in column major order. Hence we get a factor of N.
- **Array y:** It will have misses only during the first iteration of loop **i**. Hence we get a factor of 1.

Problem 4

System details

1. **PAPI version:** 7.1.0.0
2. **L1 Data Cache:**
 - Total Size = 32 KB
 - Line Size = 64 B
 - Number of Lines = 512
 - Associativity = 8
3. **L1 Instruction Cache:**
 - Total Size = 32 KB
 - Line Size = 64 B
 - Number of Lines = 512
 - Associativity = 8
4. **L2 Unified Cache:**
 - Total Size = 256 KB
 - Line Size = 64 B
 - Number of Lines = 4096
 - Associativity = 4
5. **L3 Unified Cache:**
 - Total Size = 12288 KB
 - Line Size = 64 B
 - Number of Lines = 196608
 - Associativity = 16

Speed Up Formula

Speed up of configuration i w.r.t non-blocking execution (T_{NB}) is given by

$$S_i = \frac{T_{NB}}{T_i}$$

where T_i = Time taken in the i^{th} configuration using blocking.

Table 6

Serial No.	A	B	C	T_{NB} (μ s)	T_i (μ s)	Speed Up (S_i)
1	64	64	64	42322943	10901423	3.88
2	32	32	32	42632611	8393560	5.08

3	16	16	16	43150586	57178520	7.54
4 (W)	4	4	4	43510263	16983324	2.56
5	64	64	32	37675544	7927337	4.75
6 (B)	64	32	16	35707636	4590873	7.78
7	32	16	4	41905813	6804918	6.15
8	16	16	4	42821161	6958965	6.15

(B) - Best Speed Up; (W) - Worst Speed Up

Based on the test cases above, it is concluded that the configuration [64,32,16] gives the best speed up.

Following are the values of the PAPI counters for this configuration:

Table 7

Serial No.	Counter Name	Counter Value for Best Case	Counter Value for Worst Case
1	PAPI_TOT_INS	135654702307	167696040879
2	PAPI_TOT_CYC	179805634405	269729829357
3	PAPI_L1_TCM	16091890331	13023821336
4	PAPI_L2_TCM	18265327689	16371939599